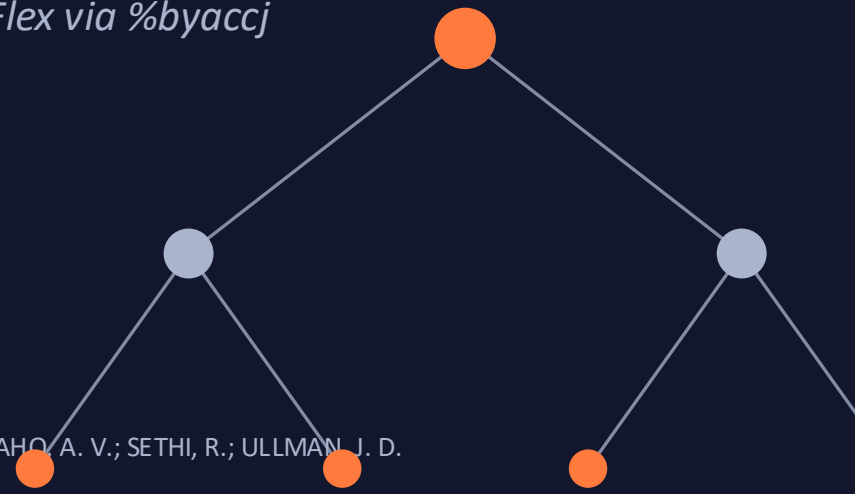
CONSTRUÇÃO DE COMPILADORES · PUCRS

Módulo 8 · Geradores de Analisadores

ByaccJ na Prática

Geração de parsers LALR(1) para Java, do arquivo .y ao executável — integrado com JFlex via %byaccj



Baseado no material do Módulo 8 (Geradores de Analisadores) e no tutorial ByaccJ/JFlex do Prof. Júlio Machado. Referência principal: AHO, A. V.; SETHI, R.; ULLMAN, J. D. Compiladores: Princípios, Técnicas e Ferramentas, 2ed.

Agenda

8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

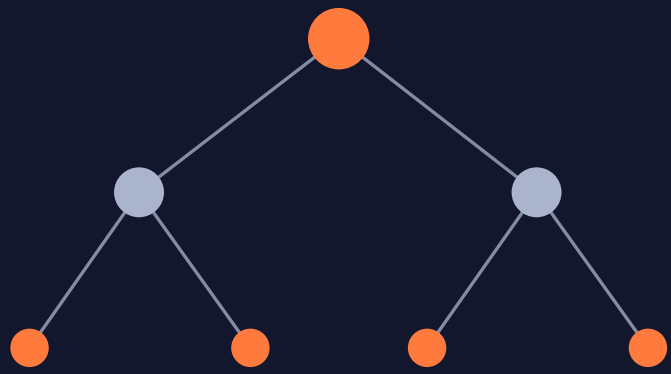
8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

8.2.1

Visão geral: Yacc, Bison, Byacc e ByaccJ



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

DEFINIÇÃO

Como funcionam os geradores de compiladores

- Geradores de compiladores produzem partes de um compilador a partir de uma especificação concisa: scanner, parser, gerador de código, otimizador...
- Especificação léxica (ex.: gramática regular) → gerador de scanner → scanner.
- Especificação sintática (ex.: gramática de atributos) → gerador de parser → parser.
- Scanner + parser + classes do usuário (tabela de símbolos, gerador de código, programa principal) são compilados e ligados no compilador final.

ferramenta	gera	linguagens
Yacc / ByaccJ	parser LALR(1)	C / Java
Lex / JFlex	scanner	C / Java / C#
Coco/R	scanner + parser	Java, C#, Modula-2...

```
spec. -> gerador -> peça do compilador

pipeline de um gerador de compilador

especificação léxica (regex)
   -> gerador de scanner -> scanner

especificação sintática (atributos)
   -> gerador de parser -> parser

classes do usuário:
  - tabela de símbolos
  - gerador de código
  - programa principal

scanner + parser + classes do usuário
  -> compilador & linker
  -> compilador gerado
```

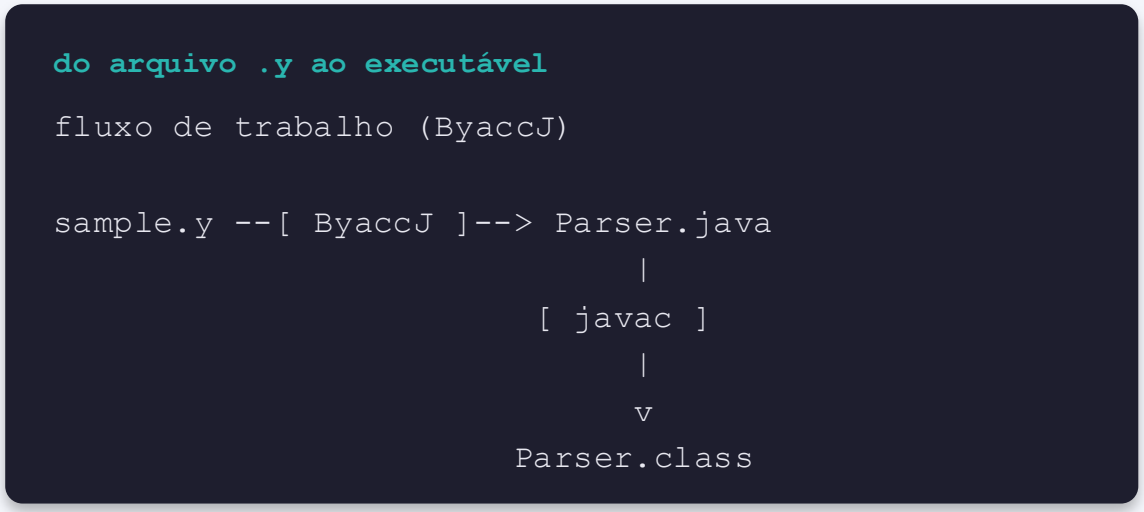
NOTA

Yacc, Bison, Byacc e ByaccJ

- Yacc ("Yet Another Compiler Compiler") nasceu em 1975 no Bell Labs, junto com C e o Unix — gera parsers LALR(1).
- GNU Bison é a versão GNU do Yacc (saída em C/C++); Byacc é o Berkeley Yacc.
- ByaccJ (Byacc/J) é a extensão do Berkeley Yacc que gera código Java em vez de C — a ferramenta usada neste material.
- Assim como o parser LALR(1) construído "na mão" no Módulo 4, o ByaccJ gera automaticamente as tabelas Action/Goto a partir da gramática — só que prontas para uso em Java.

do arquivo .y ao executável

fluxo de trabalho (ByaccJ)

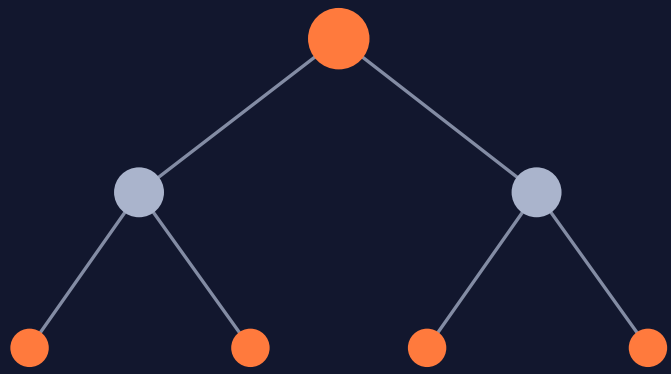


Por que ByaccJ, e não Bison?

- Nosso projeto de compilador desta disciplina é implementado em Java — precisamos de um gerador LALR(1) que produza classes Java diretamente.
- ByaccJ gera um arquivo Parser.java completo, com pilha, tabelas de análise e o método yyparse() já implementados.
- Ele foi projetado para trabalhar junto com o JFlex (mesma dupla usada no Módulo 8.3) — daremos ênfase a essa integração (%byaccj) mais adiante.
- Compare com a tabela Action/Goto que construímos manualmente no Módulo 4: é exatamente esse tipo de tabela que o ByaccJ gera por trás dos panos.

8.2.2

Estrutura do arquivo .y



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

DEFINIÇÃO

As três seções do arquivo .y

esqueleto de um arquivo .y

```
%{
   ... pacote e imports Java ...
%}
... declarações (tokens, precedência) ...

%%
... gramática e ações semânticas ...
%%

... código do usuário (Java) ...
```

- Duas linhas "%%" separam três grandes seções: declarações, gramática e ações, código do usuário.

O que vai em cada seção

- Declarações: tokens (%token), regras de precedência/associatividade e código Java entre %{ e %} (ex.: referências a pacotes necessários).
- Gramática e ações: as regras de produção da GLC que definem a linguagem, cada uma associada ao código Java a executar quando a regra é usada.
- Código do usuário: métodos obrigatórios (yyerror, yylex) e qualquer outro atributo ou método auxiliar (ex.: main).

EXEMPLO

Seção de declarações

exemplo de declarações

```
%{
  import java.util.*;
%}

%token number
%left '+'
%left '*'
```

- %token: declara tokens que não são strings literais — devem ser reconhecidos pelo analisador léxico (JFlex).
- %left / %right / %nonassoc: listam tokens (um por comando) em ordem crescente de precedência, definindo associatividade à esquerda, à direita ou nenhuma.

Convenção de numeração de tokens

- eof == 0
- Caracteres literais recebem seu valor ASCII (ex.: '+' == 43).
- YYERRCODE == 256.
- Os demais tokens são numerados a partir de 257, na ordem declarada — acessados pelo nome tanto na gramática quanto em yylex().

DEFINIÇÃO

Seção de gramática: convenções

formato de uma produção

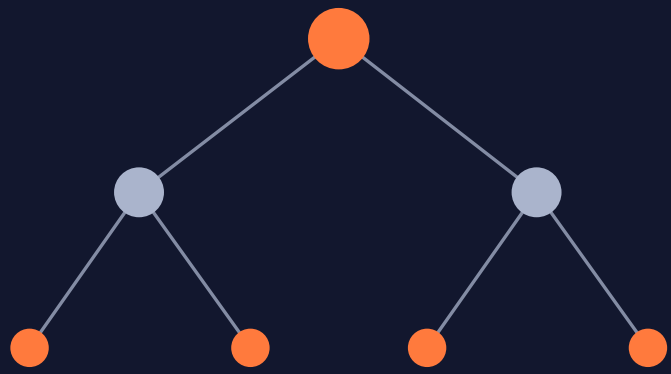
```
NT : alternativa { '|' alternativa } ';'
alternativa : { NT | T } [ '{' ... ação Java ... '}' ]
```

elemento	convenção
: e ;	dois-pontos separa NT do lado direito; ponto-e-vírgula termina a regra
|	agrupa alternativas com o mesmo lado esquerdo
'x'	terminal entre aspas simples (literal) ou símbolo de token já declarado
palavra sem aspas	se não for um token, é uma variável (não-terminal) da gramática

```
expr: term  { $$ = $1; }
    | expr '+' term  { $$.ival = $1.ival + $3.ival; }  ;
```

8.2.3

Atributos semânticos e a classe ParserVal



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

DEFINIÇÃO

$$ e $i: atributos sintetizados

acesso aos atributos numa ação semântica

```
expr: term              { $$ = $1; }
    | expr '+' term     { $$.ival = $1.ival + $3.ival; }
    ;
```

- $$ referencia o atributo do não-terminal do lado esquerdo da regra.
- $i referencia o atributo do i-ésimo símbolo do lado direito ($1 = primeiro símbolo, $2 = segundo, …).
- Ações semânticas só podem aparecer no fim de uma alternativa e podem conter qualquer código Java.
- Como vimos no Módulo 4: parsers ascendentes calculam naturalmente atributos sintetizados — cada $$ é montado a partir dos $i já reduzidos na pilha (o ByaccJ implementa isso com uma pilha de atributos por trás de $$/$i).

EXEMPLO

A classe ParserVal

```
modelo dos valores semânticos

class ParserVal {
  int ival;       // valor int
  double dval;    // valor double
  String sval;    // valor String
  Object obj;     // valor genérico

  ParserVal(int val)     {...}
  ParserVal(double val)  {...}
  ParserVal(String val)  {...}
  ParserVal(Object val)  {...}
}
```

- Todo token e todo não-terminal carrega um atributo do tipo ParserVal — valores mais complexos vão em obj.

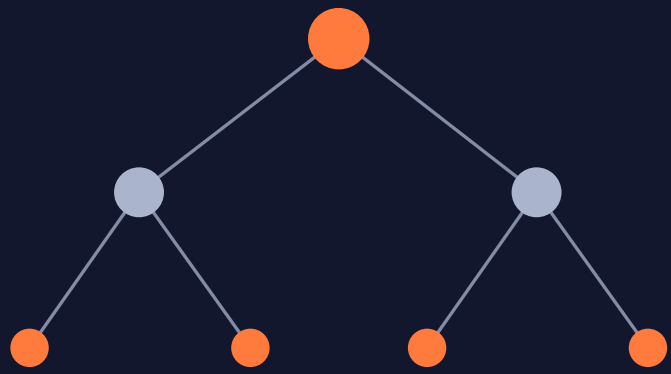
Como o valor chega até $1, $2...

```
// no scanner (yylex):
yylval = new ParserVal(n);
```

- O scanner devolve o atributo do token na variável global yylval, como um ParserVal.
- Para não-terminais, cada atribuição a $$ empilha o atributo na pilha de atributos; os acessos a $1, $2... desempilham o valor correspondente — coerente com a pilha do parser LR do Módulo 4.

8.2.4

Precedência, associatividade e tratamento de erros



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

EXEMPLO

Precedência e associatividade na prática

gramática estratificada já define a precedência

```
expr: term | expr '+' term ;
term: factor | term '*' factor ;
factor: number | '(' expr ')' ;
```

declarando precedência explicitamente (%left/%right)

```
%token number
%left  '+'
%right '^'      // potência: assoc. à direita
%left  '*'
%%
expr: number
    | expr '+' expr  { $$.ival = $1.ival + $3.ival; }
    | expr '*' expr  { $$.ival = $1.ival * $3.ival; }
    | expr '^' expr  { $$.ival =
        (int)Math.pow($1.ival,$3.ival); }
    | '(' expr ')'   { $$ = $2; }
    ;
```

- A gramática estratificada (expr/term/factor) já embute a precedência: '*' tem mais força que '+' porque está "mais perto" das folhas.
- Alternativa: gramática mais simples e plana (expr: expr '+' expr | expr '*' expr | ...), com %left/%right resolvendo a ambiguidade.
- Operadores são declarados em ordem crescente de prioridade — na tabela ao lado, '*' tem mais precedência que '+' (declarado depois).
- %right é o caso prático de um operador como potência (^) ou atribuição (=), que se associam da direita para a esquerda: 2^3^2 == 2^(3^2).
- Sem essas declarações, a gramática plana geraria conflitos shift-reduce — exatamente o tipo discutido no Módulo 4.

ATENÇÃO

Tratamento de erros: a regra error

```
alternativa error

Statement: ...
         | ...
         | error ';' ;
         // sincroniza no próximo ';'
```

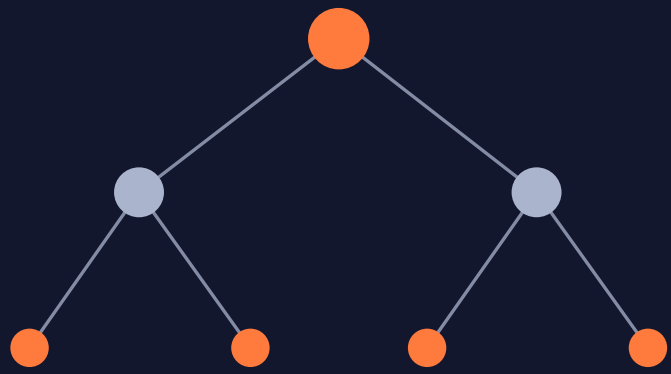
- Para não-terminais como Statement/Expression, o programador declara alternativas com o token especial error.
- Ao detectar um erro em A, o parser desempilha estados até um em que "shift error" seja válido.
- Em seguida descarta tokens da entrada até reconhecer uma sequência redutível a α (ex.: até o próximo ';').
- Por fim reduz error α para A e executa a ação semântica associada — retomando a análise normalmente.

Relação com o Módulo 4

- Esta é exatamente a estratégia de "modo pânico" vista no Módulo 4: descarta símbolos e desempilha estados até achar um ponto de sincronização.
- A diferença é que aqui o ByaccJ já oferece a sintaxe pronta (error) para declarar esse ponto — sem precisar programar a recuperação manualmente.
- Boa prática: colocar error ';' (ou error '}') nos não-terminais de comando, para que um erro em uma linha não derrube o resto do arquivo.

8.2.5

Integração com JFlex (%byaccj)



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

DEFINIÇÃO

O arquivo .flex para ByaccJ

```
seção de declarações do .flex
%byaccj                        // obrigatório: gera scanner compatível com ByaccJ

%{
  private parser yyparser;    // referência ao parser gerado pelo ByaccJ
  public Yylex(java.io.Reader in, parser yyparser) {
    this(in);
    this.yyparser = yyparser;
  }
%}
```

- %byaccj é obrigatório na seção de declarações — instrui o JFlex a gerar um scanner compatível com o parser do ByaccJ.
- É preciso incluir código Java que referencie um objeto da classe do parser: um atributo (yyparser) e um novo construtor que o recebe por parâmetro.
- Na seção de regras, cada expressão regular deve retornar o token correspondente (conforme os tokens declarados no arquivo .y).
- O valor semântico de cada token é repassado através do atributo yylval do próprio objeto scanner — que deve receber um ParserVal.

EXEMPLO

Exemplo: scanner conectado ao parser

trecho de calc.flex (seção de regras)

```
%byaccj
%%

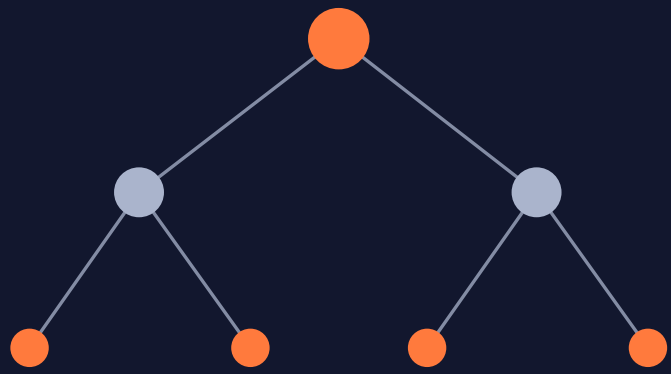
[0-9]+ {
  yylval = new ParserVal(Integer.parseInt(yytext()));
  return parser.number;      // token declarado no .y
}

"+"    { return '+'; }
"*"    { return '*'; }
"("    { return '('; }
")"    { return ')'; }
[ \t\r\n]+   { /* ignora espaços em branco */ }
```

- Cada regra combina o reconhecimento léxico (regex) com a devolução do token e, quando necessário, o preenchimento de yylval com um novo ParserVal.
- Note o uso de parser.number: como o token foi declarado no .y, o ByaccJ gera uma constante pública com esse nome, acessível a partir do scanner.

8.2.6

Exemplo completo: calculadora



8.2.1 Visão geral: Yacc, Bison, Byacc e ByaccJ

8.2.2 Estrutura do arquivo .y

8.2.3 Atributos semânticos e a classe ParserVal

8.2.4 Precedência, associatividade e tratamento de erros

8.2.5 Integração com JFlex (%byaccj)

8.2.6 Exemplo completo: calculadora

EXEMPLO

Gramática completa: calc.y

calc.y — declarações e gramática

```
%{
  import java.io.*;
%}
%token number
%left '+'
%left '*'
%%
input : expr    { System.out.println("= " + $1.ival); } ;
expr  : term    { $$ = $1; }
      | expr '+' term { $$.ival = $1.ival + $3.ival; }
      ;
term  : factor   { $$ = $1; }
      | term '*' factor { $$.ival = $1.ival * $3.ival; }
      ;
factor: number   { $$ = $1; }
      | '(' expr ')' { $$ = $2; }
      | error ';'    { yyerror("expressao invalida"); }
      ;
```

calc.y — código do usuário

```
%%

private Yylex lexer;

int yylex() {
  return lexer.yylex();
}

void yyerror(String msg) {
  System.out.println("erro: " + msg);
}

public static void main(String[] args)
    throws IOException {
  parser p = new parser();
  p.lexer = new Yylex(
    new InputStreamReader(System.in), p);
  p.yyparse();
}
```

EXEMPLO

Compilando e executando

fluxo de comandos

```
# 1) gera o parser Java a partir da gramática
$ byaccj -J calc.y          # produz parser.java

# 2) gera o scanner Java a partir das regras léxicas
$ jflex calc.flex           # produz Yylex.java

# 3) compila tudo
$ javac *.java

# 4) executa
$ java parser
2 + 3 * 4
= 14
```

- Este é o mesmo pipeline do slide "Como funcionam os geradores de compiladores" (8.2.1), agora com nomes de arquivo reais: calc.y e calc.flex entram, parser.class sai.
- As flags -Jclass=, -Jpackage= (entre outras) permitem personalizar o nome e o pacote da classe gerada — úteis para organizar um projeto Maven/Gradle.

NOTA

Síntese

- ByaccJ é a extensão Java do Berkeley Yacc: a partir de um arquivo .y, gera um parser.java LALR(1) completo (pilha, tabelas, yyparse()).
- Um arquivo .y tem três seções: declarações (tokens, %left/%right/%nonassoc), gramática e ações (com $$ e $i), e código do usuário (yyerror, yylex, main).
- Atributos semânticos trafegam pela classe ParserVal (ival, dval, sval, obj); o scanner os entrega via yylval.
- A integração com JFlex exige %byaccj no arquivo .flex, uma referência ao parser e o preenchimento de yylval a cada token reconhecido.
- O fluxo prático — byaccj -J, jflex, javac — é a mesma ideia de pipeline de geradores vista na abertura do módulo, agora de ponta a ponta com um exemplo real.

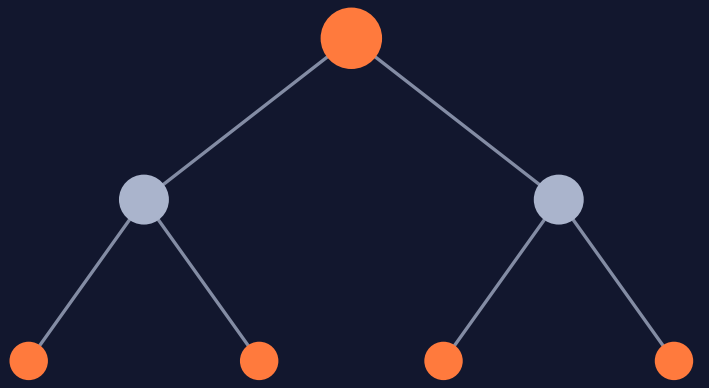
EXERCÍCIOS

Lista de exercícios

01	BÁSICO	Escreva a seção de declarações de um arquivo .y para uma calculadora que reconhece os tokens number, '+' , '-' , '*' e '/' .
02	BÁSICO	Reescreva a regra factor: number para também aceitar um número negativo unário, ajustando a ação semântica com ParserVal.
03	INTERMEDIÁRIO	Adicione o operador de potência (^) à gramática da calculadora, com %right, e implemente a ação semântica correspondente.
04	INTERMEDIÁRIO	Escreva o trecho do arquivo .flex (com %byaccj) que reconhece números decimais (com ponto) e os devolve como ParserVal do tipo dval.
05	AVANÇADO	Adicione uma regra error ';' ao não-terminal de comando da calculadora e descreva, passo a passo, o que o parser faz ao encontrar uma entrada inválida como "2 + * 3;".

Gabarito

Respostas comentadas dos exercícios de ByaccJ



Gabarito — exercícios 01 a 03

```
01) declarações

%token number
%left '+' '-'
%left '*' '/'
```

- 02) factor: number | '-' number { $$.ival = -$2.ival; } | '(' expr ')' { $$ = $2; } ; — o menos unário é reconhecido como um novo caso de factor, e o valor final é o negativo do atributo do number.
- 03) %right '^' deve ser declarado com precedência maior que '*' (linha mais abaixo na lista %left/%right); a regra expr: expr '^' expr { $$.ival = (int) Math.pow($1.ival, $3.ival); } ; usa Math.pow para calcular a potência.

Gabarito — exercícios 04 e 05

04) reconhecendo decimais em .flex

```
[0-9]+\.[0-9]+  {
  yylval = new ParserVal(Double.parseDouble(yytext()));
  return parser.number;
}
```

- 05) Com "2 + * 3;" o parser empilha 2, reduz a number/expr, empilha '+', mas ao ler '*' não há shift nem reduce válidos: erro detectado.
- O parser desempilha estados até um em que "shift error" seja válido para o não-terminal de comando, empilha o token error e descarta tokens da entrada (aqui, o '*' problemático) até encontrar ';'.
- Com error ';' completo na pilha, reduz para o não-terminal de comando e executa yyerror("expressao invalida"), retomando a análise normalmente após o ';'.